

## 2025 中国研究生创“芯”大赛•EDA 精英挑战赛

赛题六：  
模拟电路自动尺寸优化设计  
基于 PyAether 生态共塑 EDA 未来

### 作品设计报告

参赛队 ID：EDA250609

团队名称：未名湖人队

二〇二五年十一月

## 目录

|                                    |    |
|------------------------------------|----|
| 一、赛题自测成果.....                      | 3  |
| 1.1 赛题成果： .....                    | 3  |
| 1.1.1 Public case1 结果.....         | 4  |
| 1.1.2 Public case2 结果.....         | 6  |
| 二、技术路线.....                        | 7  |
| 2.1 研究基础.....                      | 7  |
| 2.2 技术方案概述.....                    | 8  |
| 2.3 参数约减算法比较与选择.....               | 8  |
| 2.4 多目标评估算法比较与选择.....              | 9  |
| 2.5 尺寸优化算法比较与选择.....               | 10 |
| 三、实施方案.....                        | 11 |
| 3.1 实施方案概述.....                    | 11 |
| 3.2 具体实施文件/程序说明.....               | 11 |
| 3.2.1 程序文件说明.....                  | 11 |
| 3.2.2 软件架构.....                    | 12 |
| 3.2.3 程序运行流程.....                  | 13 |
| 3.2.4 使用第三方库的功能及版本与路径.....         | 14 |
| 3.3 遇到问题&解决方案.....                 | 14 |
| 3.3.1 挑战一：仿真次数与参数维度的矛盾.....        | 14 |
| 3.3.2 挑战二：优化算法在效率与全局性之间的权衡.....    | 15 |
| 3.3.3 挑战三：硬性约束下优化目标的“全有或全无”困境..... | 16 |
| 四、总结与展望.....                       | 16 |
| 4.1 总结.....                        | 16 |
| 4.2 展望.....                        | 17 |
| 五、参考文献.....                        | 18 |

## 一、赛题自测成果

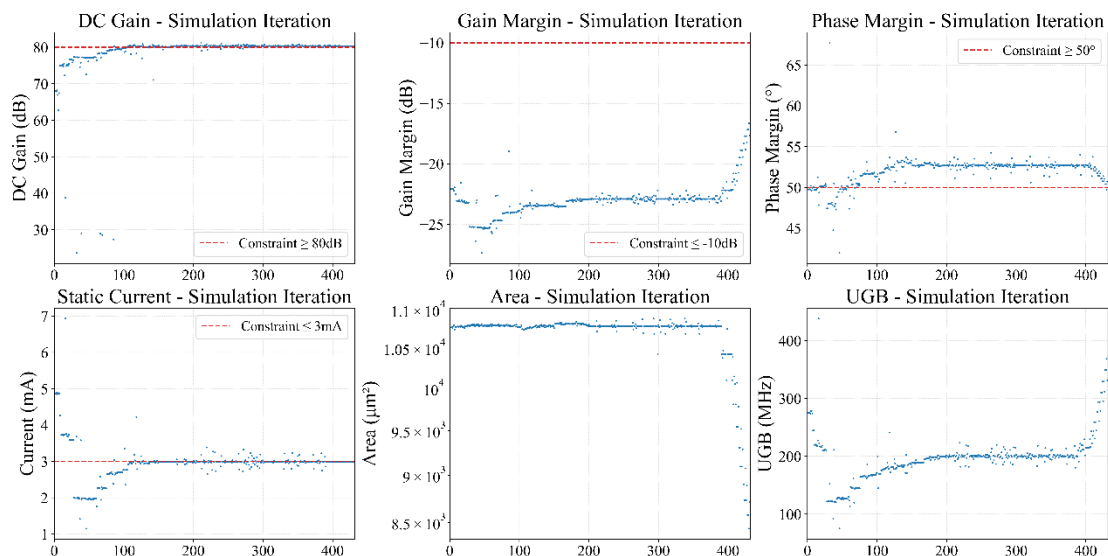
### 1.1 赛题成果：

| 类别       | 参数    | Case 1<br>(Nominal) | Case 1<br>(PVT 最差) | Case 2<br>(Nominal) | Case 2<br>(PVT 最差) | 单位              |
|----------|-------|---------------------|--------------------|---------------------|--------------------|-----------------|
| 基础<br>信息 | 初始    | 193                 |                    | 113                 |                    | 个               |
|          | 优化后   | 140                 |                    | 85                  |                    | 个               |
|          | 迭代次数  | 430                 |                    | 567                 |                    | 次               |
| 约束<br>参数 | Gain  | 82.9                | 80.0               | 60.3                | 50.0               | dB              |
|          | PM    | 55.1                | 50.2               | 87.6                | 80.5               | deg             |
|          | GM    | -22.2               | -18.8              | -11.4               | -10.1              | dB              |
|          | I_OPA | 2.74                | 2.99               | 0.689               | 0.904              | mA              |
|          | 约束满足  | 4                   | 4                  | 3                   | 3                  | 个               |
| 优化<br>目标 | UGB   | 425.6               | 287.7              | 73.1                | 33.8               | MHz             |
|          | 面积    | 9133.5              | 9133.5             | 2823.3              | 2823.3             | um <sup>2</sup> |

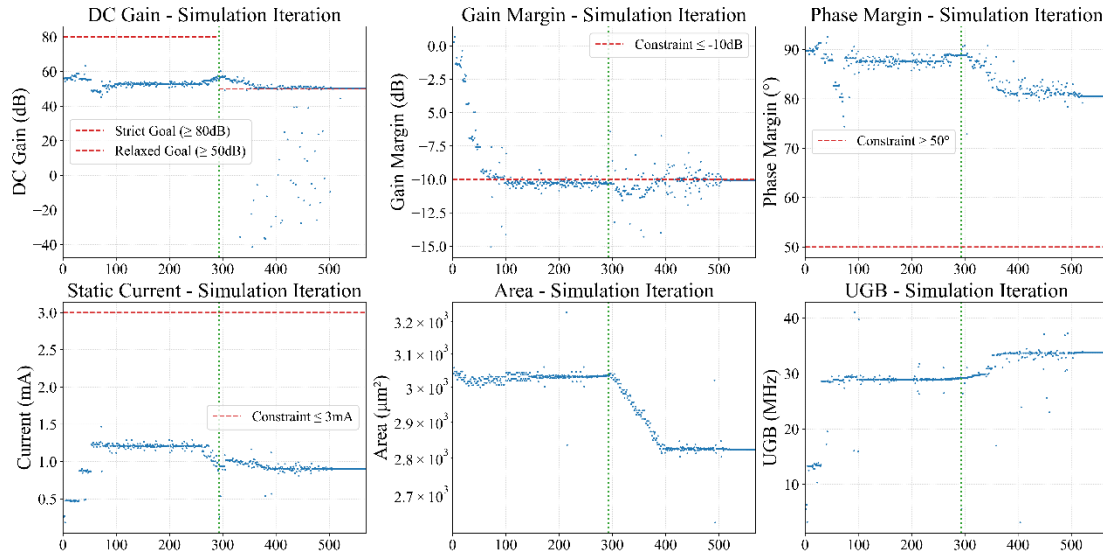
注意：这里只给出迭代次数，在开发服务器上早期测量一次 9-corner 仿真约为 13s，简单相乘得到 Case1 用时约 93min，Case2 用时约 123min。（后续在评分服务器上 Case1 用时 101min，Case2 用时 145min）

最前面给出开发服务器上自己测试的各种仿真结果和迭代次数的图：

#### Case1:



## Case2:



### 1.1.1 Public case1 结果:

约减思路：根据 schematic 中的连接关系建立一个图，然后使用 WL（Weisfeiler-Lehman）算法识别对称性，这里给出终端输出的结果。

## Case1:

Symmetry analysis found 22 groups for optimization.

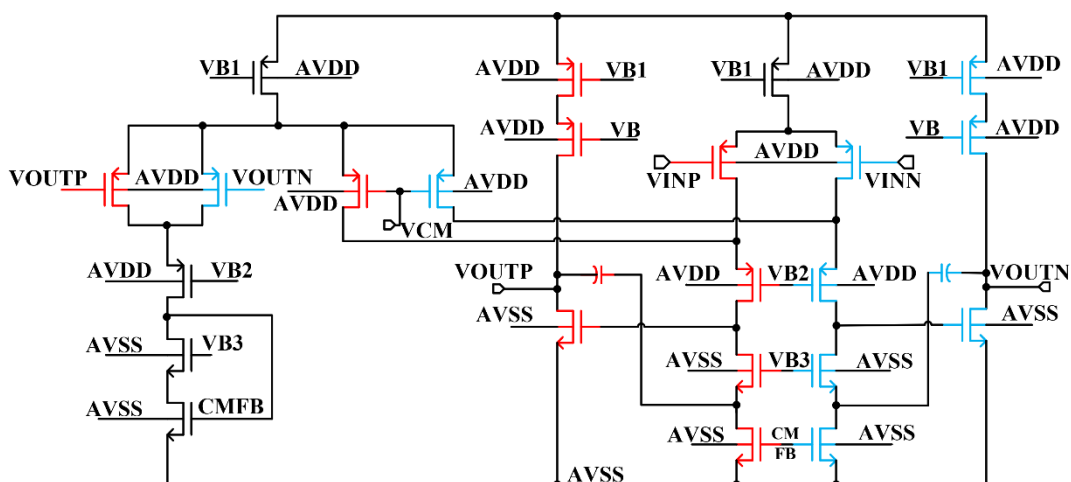
- Group 1: [NM5, NM4]
- Group 2: [NM18, NM3]
- Group 3: [NM20, NM21]
- Group 4: [NM12, NM22]
- Group 5: [NM23, NM25]
- Group 6: [NM24, NM26]
- Group 7: [NM2, NM1]
- Group 8: [PM21, PM22]
- Group 9: [PM16, PM23]
- Group 10: [PM24, PM26]
- Group 11: [PM25, PM27]
- Group 12: [PM0, PM1]
- Group 13: [PM6, PM4]
- Group 14: [PM3, PM5]
- Group 15: [R0, R1]
- Group 16: [R2, R3]
- Group 17: [R4, R5]
- Group 18: [C3, C2]
- Group 19: [C6, C7]
- Group 20: [C4, C5]
- Group 21: [R8, R6]
- Group 22: [I6, I7, I8, I9]

## Case2:

Symmetry analysis found 10 groups for optimization.

- Group 1: [PM0, PM1]
- Group 2: [NM1, NM0]
- Group 3: [NM3, NM4]
- Group 4: [PM2, PM3]
- Group 5: [PM4, PM5]
- Group 6: [NM5, NM6]
- Group 7: [C0, C1]
- Group 8: [PM26, PM24]
- Group 9: [PM15, PM23]
- Group 10: [PM9, PM7]

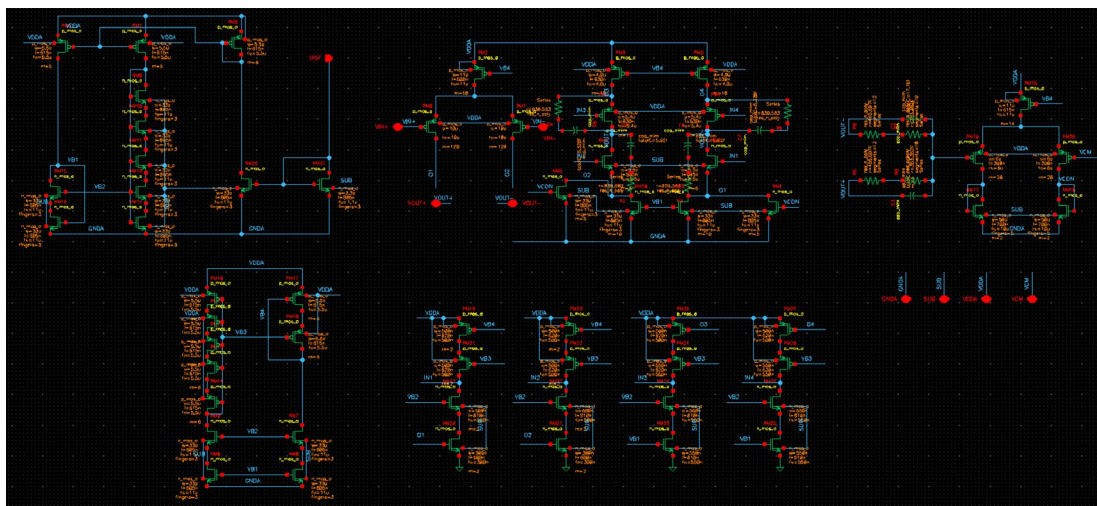
所有器件都会识别对称性，这里给出 case2 的部分电路的对称性识别结果示意



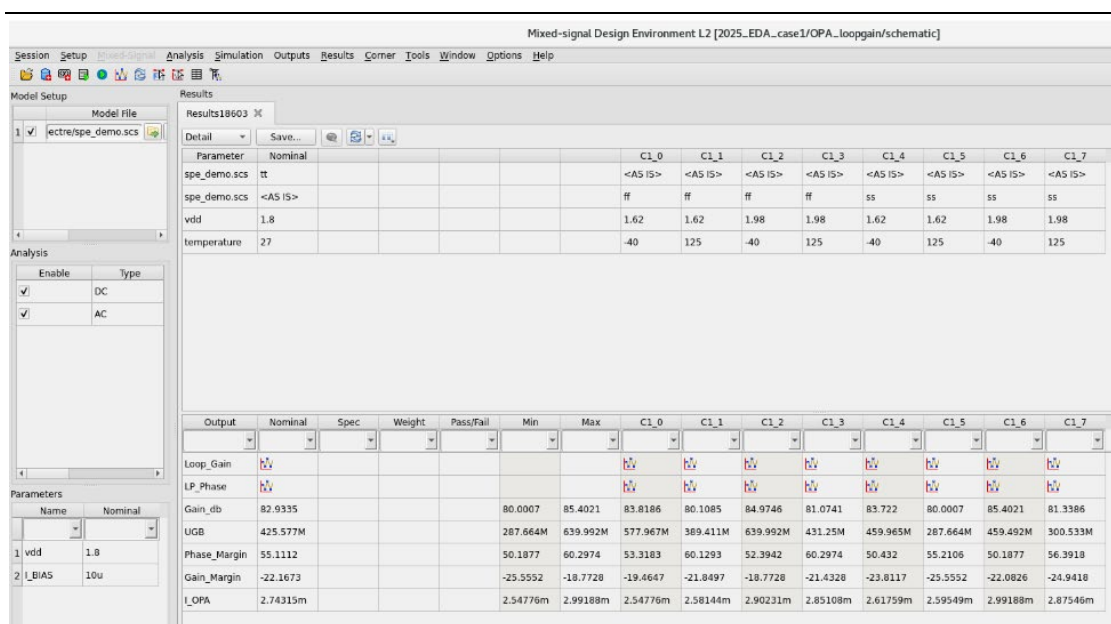
这里红色和最近的蓝色的相同器件对称，可以看到基本符合模拟电路基本对称性。

注：case1 实在有点多，schematic 图也很大，标注很难标清楚也很难看清楚，这里省略，还请见谅。

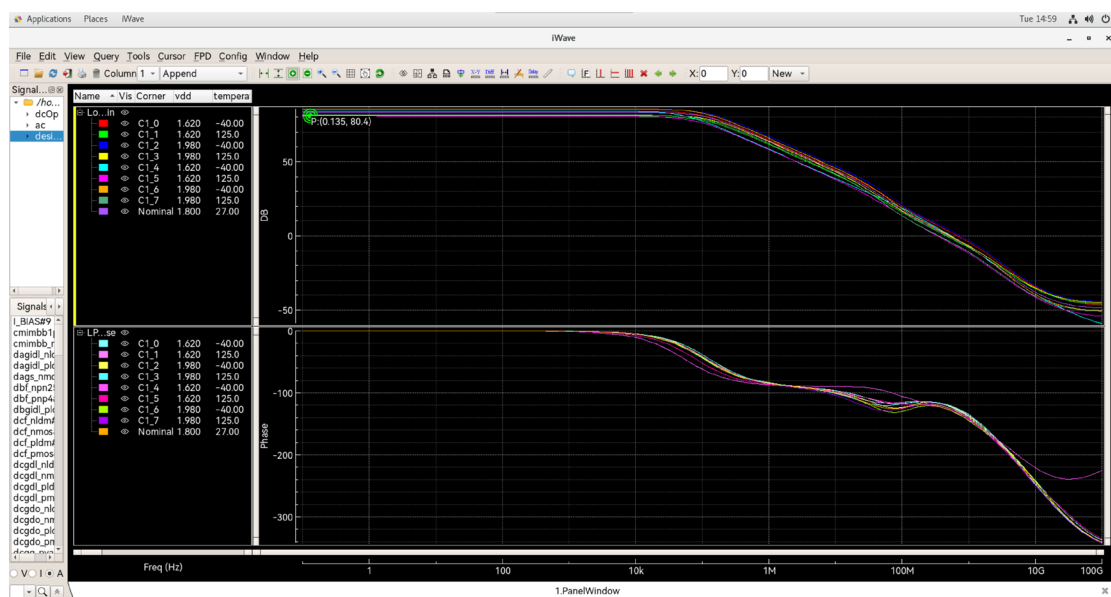
最优尺寸下的反标后电路图内部 schematic 截图（真的很难看清楚。这里给一个 case1 的截图，几乎是能看到参数的前提下放到最大了）



PVT 角仿真验证报告：关键性能指标在 Nominal 角以及八个边界 PVT 组合角 (C1\_0 至 C1\_7) 下的仿真结果如表所示。这些角涵盖了 ff、ss 等工艺角，-40°C、125°C 的温度范围和 1.62V、1.98V 的电源电压范围。



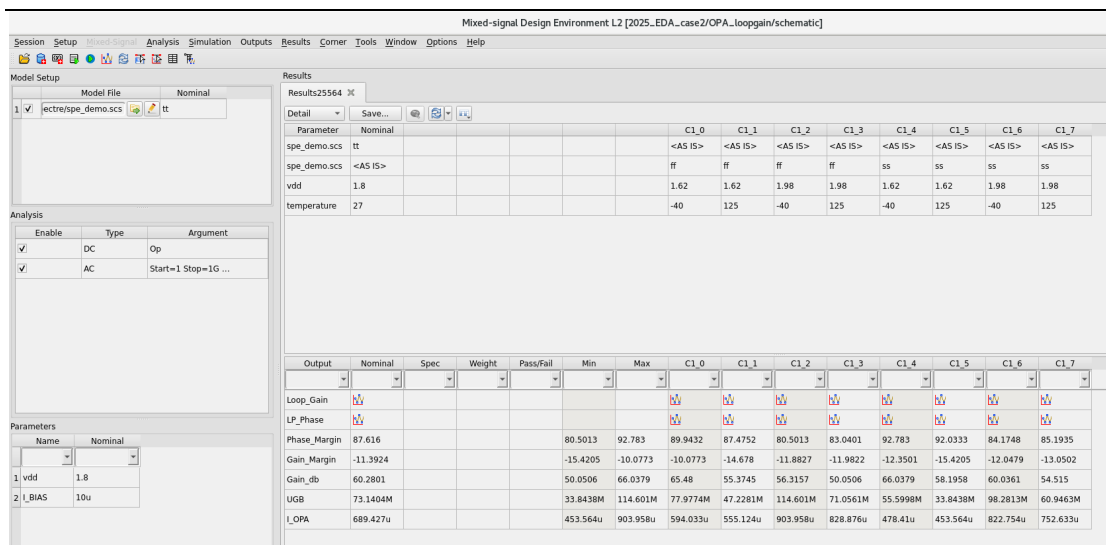
iWave 截图：可以看到 DC Gain 都高于 80dB，并且 gain 图像在相当大的频率内还算稳定。Phase 曲线在高频段存在部分非线性，这反映了算法在优化 UGB 与面积时，对稳定性进行的权衡。尽管如此，最终的 PM 和 GM 仍在所有 PVT 角点下满足设计约束。



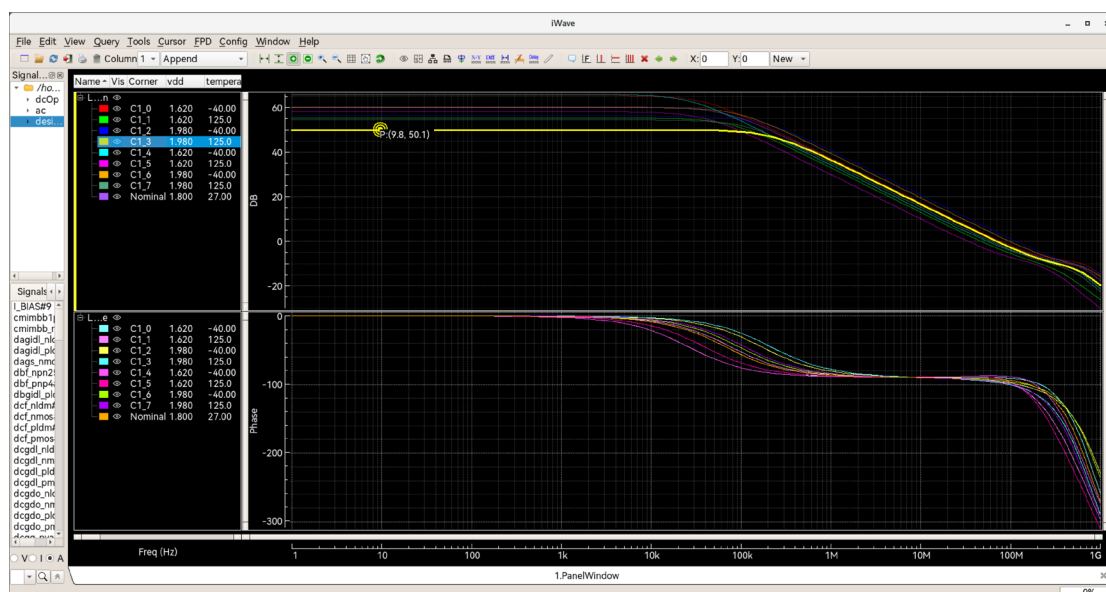
注：这里所有的 corner 设置都没有修改，使用的 mde 相关文件就是最原始的版本，故没有给出。

### 1.1.2 Public case2 结果：

PVT 角仿真验证报告



iWave 截图：可以看到曲线比较标准，DC 增益值相对较低，同时 PM 有较大余量。这表明当前设计在稳定性上表现稳健，但在增益性能上仍有进一步优化的潜力。



## 二、技术路线

### 2.1 研究基础

现代模拟集成电路设计复杂度日益增加，传统手动调优方法效率低下，且难以应对多工艺、电压和温度（PVT）角点的性能一致性挑战。因此，电路自动尺寸优化（Automated Sizing）已成为 EDA 领域的研究热点。

现有的自动化方法大致可分为三类：

基于优化算法的方法：包括模拟退火、遗传算法等。近年来，研究人员也探索了爬山算法等启发式搜索方法，它们实现简单且在局部寻优上表现高效。

基于机器学习/人工智能的方法：以强化学习（Reinforcement Learning, RL）为代表的技术展现了巨大潜力。例如，Settaluri 等人<sup>[1]</sup>成功应用 RL 来自动设计模拟电路。但这类方法通常需要海量的仿真数据（至少数万次）进行模型训练，且在面对参数维度激增或电路拓扑变化时，其可扩展性和泛化能力仍面临挑战。

基于知识/方程的方法：例如基于  $g_m/i_d$  的设计方法，它们依赖于精确的器件模型和设计者的先验知识，虽然设计速度快，但通用性有限。

考虑到竞赛场景下仿真资源有限、时间限制以及电路具体结构未知等因素，我们选择了一种高效的启发式搜索算法作为核心，并通过引入电路拓扑分析来提升其在解决高维优化问题时的效率。

## 2.2 技术方案概述

本方案提出了一套高效的模拟电路自动尺寸优化框架，其核心思想是“降维”与“分治”。

降维：通过引入图论中的 Weisfeiler-Lehman 算法，将电路网表抽象为图结构，自动识别并绑定拓扑上对称的器件，显著减少独立优化变量的数量。

分治：在降维后的参数空间上，我们设计了一种鲁棒的“二次爬山”（Two-Pass Hill Climbing）优化策略。该策略首先以严格目标进行优化（Pass 1），若未能满足所有约束，则基于当前最佳结果，自适应地放宽目标启动第二轮优化（Pass 2），极大提升了寻找可行解的成功率。

效率保障：为确保在有限时间内完成大量迭代，框架底层实现了轻量级的并行仿真调度，将单次全 PVT 角点评估的耗时缩减至串行方式的三分之一左右。

## 2.3 参数约减算法比较与选择

在模拟电路优化中，参数空间的维度是影响收敛速度和寻优质量的关键因素。为了降低优化复杂度，我们首先对参数进行约减，其核心是识别并绑定电路中的对称器件。我们比较了以下两种方法：

局部拓扑指纹法（备选方案）：

原理：该方法为每个器件（如 MOS 管）创建一个“指纹”，该指纹仅描述其自身的类型以及其各个引脚（D, S, G, B）的直接连接信息。例如，两个 PMOS 管若其源极和衬底都连接到 VDD，栅极连接到同一输入，漏极连接到相似的负载，则它们的局部指纹可能相同。

问题：这种方法的主要缺陷在于缺乏全局视角。如我们的实验发现，电路输入级和输出级的两个晶体管，尽管可能拥有相似的局部连接，但它们在整个电路中的功能和拓扑位置完全不同。若仅根据局部指纹将它们错误地绑定在一起，会施加不合理的约束，严重损害电路性能，甚至导致优化无法收敛。

Weisfeiler-Lehman (WL) 算法（最终选择）：

原理：WL 算法是一种经典的图同构测试算法。它通过一个迭代的“标签传播”过程，将节点的邻域信息逐层聚合到节点自身的标签中。经过多轮迭代，每个节点的最终标签不仅包含了其自身的类型，还编码了其在整个电路图结构中的全局位置信息。

选择理由：相比局部指纹法，WL 算法能够捕捉全局拓扑信息，从而更精确、更严格地识别出真正意义上的对称器件。拥有相同最终 WL 标签的两个器件，在拓扑上是等价的。这避免了将功能不同的器件错误绑定的风险，为后续优化提供了一个正确且维度更低的搜索空间。实验证明，该方法在官方测试用例上取得了显著的降维效果（case1: 193 优化到 140, case2: 113 优化到 85），且未引入错误约束。

综上，我们选择 WL 算法作为参数约减的核心技术，因为它在保证约束正确性的前提下，提供了更可靠的降维效果。

## 2.4 多目标评估算法比较与选择

赛题要求在满足多项性能约束的前提下，实现 UGB 最大化和面积最小化的双重目标。这是一个典型的多约束、多目标的优化问题。我们选择了一种统一的惩罚函数法来构建评估函数，其设计思想如下：

将约束问题转化为无约束优化问题：该方法的核心是将所有性能约束整合进一个单一的评分值。评估函数具有两种截然不同的工作状态：

惩罚模式（当存在性能违例时）：如果任何一个性能指标（如  $\text{Phase\_Margin} < 50$ ）不

满足约束，函数会返回一个巨大的惩罚值（ $1e9$ ）并线性叠加各项约束的归一化违例量（ $\text{total\_violation} * 1e6$ ）。此时，优化器的目标是最小化惩罚值，其行为等同于全力寻找一个满足所有约束的“可行解”。

优化模式（当所有约束满足时）：一旦所有约束都得到满足，惩罚项消失，评分函数切换为对核心目标的评估。我们的评分公式为  $\text{Score} = 0.5 * \text{Area\_norm} - 0.5 * \text{UGB\_norm}$ 。其中，面积（Area）和单位增益带宽（UGB）都被归一化处理。优化器的目标依然是最小化 Score 值，这等价于最小化面积和最大化 UGB。

选择理由：

目标明确：该方法为爬山等启发式算法提供了一个清晰、单一的优化方向（始终是最小化 Score），实现简单且高效。

优先级自动切换：函数本身的设计内嵌了“先满足约束，再优化目标”的逻辑，无需在算法外部进行复杂的流程控制。

计算开销低：与其他多目标优化方法（如帕累托前沿法）相比，单点评估的计算开销极低，适合在仿真次数受限的场景下使用。

## 2.5 尺寸优化算法比较与选择

在确定了参数空间和评估函数后，我们选择了一种经过深度改进的启发式爬山算法（Hill Climbing）作为核心优化引擎。

基础算法选择：爬山算法

相比于模拟退火、遗传算法等全局优化算法，爬山算法是一种局部搜索算法，它从一个初始解出发，迭代地向相邻的、评估分数更优的状态移动，直至到达一个局部最优解。我们选择它的理由是：实现简单、收敛速度快、计算开销小，非常适合在有限时间内进行快速迭代。

算法改进与增强

传统的爬山算法容易陷入局部最优且效率不高。为此，我们设计并实现了一系列增强机制，将其改造为一个高效、鲁棒的优化框架：

两阶段参数优化：在单次优化迭代中，我们将参数分为两类进行处理。首先集中优化对电路工作点影响较大的离散参数（如 MOS 管的  $m$ ），待其稳定后，再对连续参数（如  $I$ ，

w) 进行精细微调。这种分而治之的策略提高了优化的稳定性和效率。

参数冻结机制：在优化过程中，算法会动态追踪每个参数的有效性。如果对某个参数在多个方向上进行微扰后，评估分数均未得到改善，则该参数会被暂时“冻结”，不再对其进行尝试。这使得宝贵的仿真次数可以集中在那些对性能影响更显著的“活跃”参数上，避免了资源浪费。

贪心跳转机制：为了加速收敛，尤其是在优化初期快速脱离较差的初始状态时，我们引入了贪心跳转。在试探参数时，若某次改动带来的性能提升（即评估分数的下降）超过一个预设阈值，算法将不再继续试探其他参数，而是立即接受这个“足够好”的改动，并开始下一轮迭代。这使得优化器能沿着最陡峭的方向快速前进。

## 三、实施方案

### 3.1 实施方案概述

本方案完全基于 Python 语言实现，并深度集成大赛官方提供的 PyAether 工具链，以实现与 EDA 设计环境的交互。整个方案的核心是 `optimization.py` 主控脚本，它统一调度了拓扑分析、参数优化和并行仿真三大功能模块。

在代码设计上，我们遵循了高度模块化的原则。不同的核心功能被清晰地解耦到 `src` 目录下的各个独立模块中，例如，图结构定义与分析位于 `data_models.py` 和 `circuit_analyzer.py`，与 EDA 工具的交互及仿真被封装在 `optimizer.py` 中。这种设计不仅提高了代码的可读性和可维护性，也使得各个功能模块可以独立测试和改进。

### 3.2 具体实施文件/程序说明

#### 3.2.1 程序文件说明

我们的实现由一个主程序和 `src` 目录下的多个支持模块构成，各文件职责如下：

`optimization.py`: 主控程序。作为整个方案的入口，负责解析命令行参数、初始化仿真平台，并调用 `run_managed_optimization_flow` 函数来执行完整的“二次爬山”（Two-Pass）优化流程。

`src/optimizer.py`: 仿真平台模块。定义了核心的 `SimulatePlatform` 类，它封装了所有与 PyAether 的底层交互。该类负责参数设置、面积计算，并通过调用并行工具模块来执行高

效的多 PVT 角点仿真。

`src/parallel_utils.py`: 并行工具模块。包含了实现并行仿真的关键逻辑。

`prepare_parallel_views` 和 `cleanup_parallel_views` 函数负责创建和清理 9 个独立的 MDE 仿真环境，而 `simulation_worker` 函数则是每个子进程实际执行的仿真任务。

`src/data_models.py`: 数据结构模块。定义了用于电路拓扑分析的基石，包括 `CircuitGraph`、`DeviceNode`、`NetNode` 和 `Parameter` 等核心数据类。

`src/graph_builder.py`: 图构建模块。其 `build_graph_from_eda` 函数负责读取 PyAether 提供的电路视图信息，并将其转化为我们定义的 `CircuitGraph` 对象，为后续分析做好准备。

`src/circuit_analyzer.py`: 电路分析模块。包含了 `find_topological_symmetries` 函数，这是 Weisfeiler-Lehman (WL) 算法的具体实现，用于在 `CircuitGraph` 对象上进行对称性识别。

`src/utils.py`: 通用工具模块。包含一系列辅助函数，如 `read_parameters` 用于从文件加载参数并应用边界约束，`param_convert` 用于处理带单位的字符串数值。

### 3.2.2 软件架构

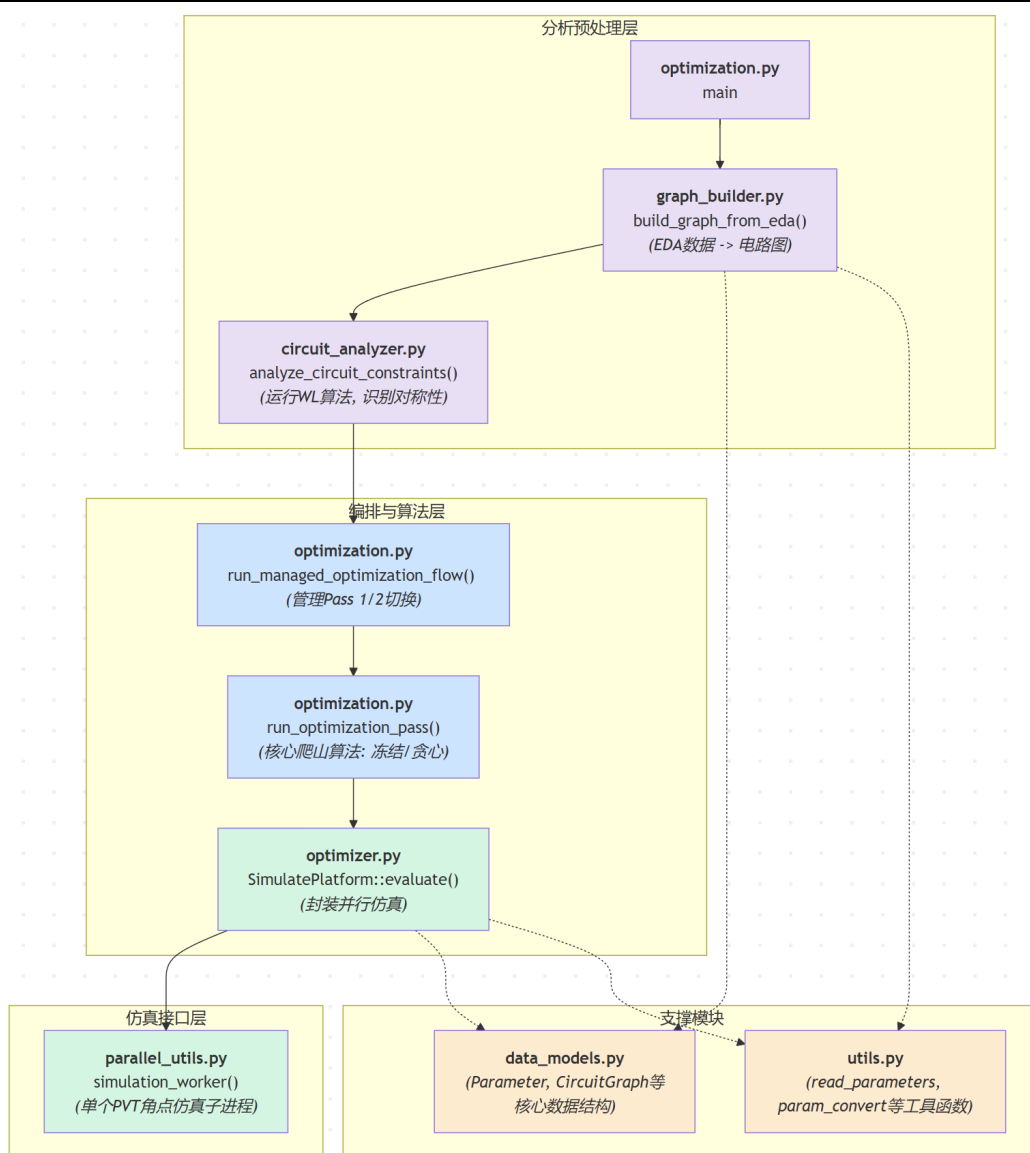
本方案的软件架构清晰地分为四个层次，各层次之间通过明确的接口进行调用，实现了高度的内聚和低耦合。

**编排层**：由 `optimization.py` 中的 `run_managed_optimization_flow` 函数实现。它作为最顶层逻辑，负责管理整个优化流程，包括决策何时从 Pass 1 切换到 Pass 2。

**优化算法层**：由 `optimization.py` 中的 `run_optimization_pass` 函数实现。这是爬山算法的核心引擎，内含参数冻结、贪心跳转等具体策略。

**仿真接口层**：由 `src/optimizer.py` 中的 `SimulatePlatform` 类实现。它为上层提供简洁的 `evaluate()` 接口，同时屏蔽了底层并行仿真的复杂实现细节。

**分析预处理层**：由 `src/graph_builder.py` 中的 `build_graph_from_eda` 和 `src/circuit_analyzer.py` 中的 `analyze_circuit_constraints` 协同工作。在优化开始前，它将电路网表转换为图结构并完成对称性分析。



### 3.2.3 程序运行流程

程序从启动到输出最终结果的完整流程如下：

初始化：optimization.py 启动，解析命令行参数，并初始化 SimulatePlatform 仿真平台。

拓扑分析：程序调用 build\_graph\_from\_eda 构建电路图，随后调用 analyze\_circuit\_constraints 运行 WL 算法，识别对称器件并建立约束组。

并行环境准备：调用 parallel\_utils.prepare\_parallel\_views，一次性创建并配置好 9 个 PVT 角点对应的独立 MDE 仿真环境。

基准评估：对初始参数进行一次全 PVT 角点并行仿真，获取性能基准。

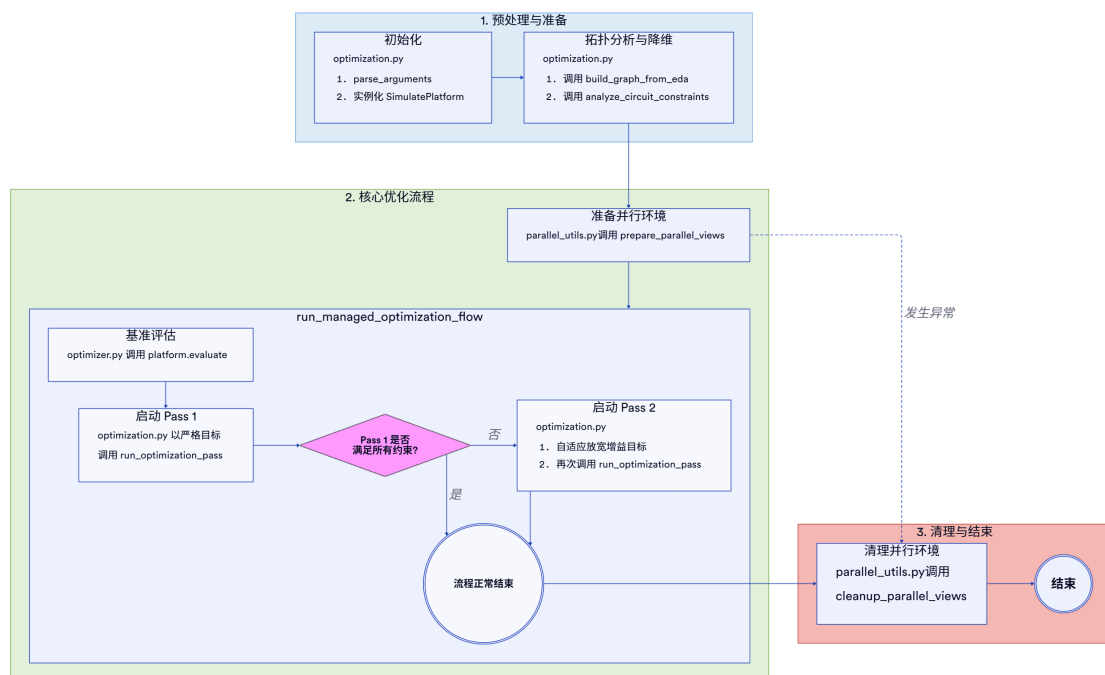
启动 Pass 1：调用 run\_optimization\_pass，以严格的增益目标（80dB）和预设的仿真预

算开始第一轮爬山优化。

检查与决策：Pass 1 结束后，检查其找到的最佳解是否满足所有约束。若满足，则流程结束；若不满足，则进入下一步。

启动 Pass 2 (松弛目标优化)：基于 Pass 1 的最佳结果，自适应地放宽增益目标，并再次调用 `run_optimization_pass`，利用剩余的仿真预算继续寻优。

环境清理与结束：在程序的 `finally` 语句块中，调用 `parallel_utils.cleanup_parallel_views` 清理所有临时仿真环境，并输出最终找到的最优参数。



### 3.2.4 使用第三方库的功能及版本与路径

只使用到了官方给出的 `numpy`、`PyAether` 以及最基本的 `python` 内置包

### 3.3 遇到问题&解决方案

我们的实施方案是一个不断迭代和演进的过程，其核心是解决在有限时间内（3 小时）应对高维、多约束优化问题时遇到的三大挑战。

#### 3.3.1 挑战一：仿真次数与参数维度的矛盾

问题描述：

项目初期面临的最大的挑战是“维度灾难”与“时间预算”的冲突。赛题给出的电路参数维度极高（Case 1: 193 个，Case 2: 113 个），而官方提供的 `NetlistAndRun` 接口是串行执

行 9 个 PVT 角点的仿真，单次全角点评估耗时约 30-45 秒（早期服务器非拥堵时期运行时间）。这意味着在有限的总时间内，我们能进行的迭代次数极其有限，这对任何优化算法都是严峻的考验。

解决方案：

我们从“开源”和“节流”两个方面同时入手，以缓解这一矛盾。

节流：通过拓扑分析降低参数维度。我们没有采用启发式或手动的方式绑定参数，而是引入了图论中的 Weisfeiler-Lehman (WL) 算法。如 3.3.1 节所述，该算法通过迭代的标签传播，能够从全局拓扑视角精确识别出真正对称的器件。

效果：该方法成功将 Case 1 的独立变量从 193 个减少到 140 个，Case 2 从 113 个减少到 85 个，在不引入错误约束的前提下，显著缩小了搜索空间。

开源：通过并行化提升仿真效率。为了突破串行仿真的瓶颈，我们设计了一套轻量级并行仿真框架。如 3.3.3 节所述，该框架利用 Python 的 multiprocessing 库，在优化开始前一次性创建 9 个独立的 MDE view，并在每次评估时启动 9 个子进程并行执行仿真，最后由主进程聚合结果。

效果：此举将单次全角点评估的耗时缩减至原先的三分之一左右，等效于将我们的“仿真预算”提升了近三倍，为后续更复杂的算法策略提供了可能性。

### 3.3.2 挑战二：优化算法在效率与全局性之间的权衡

问题描述：

尽管通过降维和并行化争取了更多迭代机会，但选择何种优化算法依然至关重要。模拟退火、差分进化等全局优化算法虽然有跳出局部最优的能力，但通常需要海量的仿真次数才能收敛，这在本场景下并不可行。传统的爬山算法虽然速度快，但效率依然不高，且容易过早收敛。

解决方案：

我们选择以爬山算法为基础，并通过引入两种增强机制来提升其搜索效率，使其更适应本赛题的特点。

参数冻结：算法会动态追踪每个参数的有效性。若对某个参数在多个方向微扰后，评估分数均未改善，则该参数会被暂时“冻结”。这避免了在不敏感的参数上浪费宝贵的仿真次数，使算力能集中在对性能影响更显著的“活跃”参数上。

贪心跳转：在优化初期，为了快速脱离较差的初始状态，若某次参数试探带来的性能

提升（惩罚项降低）超过一个预设阈值，算法会立即接受该改动并开始新一轮迭代，不再遍历本轮剩余参数。这使得优化器能沿着最陡峭的方向快速下降，迅速逼近可行解区域。

### 3.3.3 挑战三：硬性约束下优化目标的“全有或全无”困境

问题描述：

我们的评估函数设计为分层结构：只有当所有硬性约束（相位裕度、增益等）都满足时，优化器才会开始对 UGB 和面积进行优化。这种设计的初衷是保证解的合规性，但也带来了一个严重问题：如果爬山算法不幸无法满足某个硬性约束，那么 UGB 和面积将完全得不到优化。

解决方案：

为了解决这一问题，我们设计了整个框架中最具创新性的“二次爬山”（Two-Pass Hill Climbing）策略。

**Pass 1：追求理想。**优化器首先以一个严格的性能指标（如  $DC\ Gain \geq 80dB$ ）为目标进行寻优。在分配的预算内，算法会尽力寻找满足这一高标准的最优解。

**Pass 2：保障现实。**如果 Pass 1 未能找到满足所有约束的解，框架并不会宣告失败。它会记录下 Pass 1 找到的最佳结果（即使它不完全合规），并基于该结果启动 Pass 2。在这一阶段，无法满足的约束目标会被自适应地放宽到一个更“现实”的水平（例如，若 Pass 1 最高增益为 63dB，则新目标被设为 60dB）。优化器从 Pass 1 的最佳参数点出发，以这个更容易实现的目标继续寻优。

**效果：**这一调整立竿见影。一旦解除了严苛约束带来的高额惩罚，评估函数立刻进入第二层，优化器便能重新聚焦于 UGB 和面积这两个核心目标，最终交付一个完全合规的优良设计。Case 2 的成功优化过程完美验证了该策略的鲁棒性。

## 四、总结与展望

### 4.1 总结

本报告详细介绍了一种用于模拟电路自动尺寸优化的新型框架。该框架通过一个问题驱动的演进过程，系统性地解决了高维参数空间、有限仿真预算和严苛性能约束带来的多重挑战。我们首先通过 WL 图同构算法和轻量级并行仿真来缓解维度与时间的矛盾；接着采用带有参数冻结和贪心跳转机制的增强型爬山算法提升寻优效率；最终，通过创新

的“二次爬山”策略解决了硬性约束下的收敛难题。实验结果证明，该框架能够在有限时间内，为复杂电路快速、可靠地找到满足所有 PVT 约束的优质设计方案，验证了本方案的有效性与鲁棒性。

## 4.2 展望

尽管当前框架取得了成功，但仍存在进一步优化的空间，未来工作可从以下几个方面展开：

**优化搜索路径的敏感性：**我们的爬山算法在参数微扰顺序上采用了随机策略，但这可能对优化路径产生影响。实验中观察到，不同的随机顺序可能导致最终的 UGB 和面积存在差异（如 Case 1 中 UGB 在 280MHz 到 340MHz 间波动）。更关键的是，在 Case 2 中，若算法早期优先探索了正确的参数，可能将增益稳定在 60dB，从而使得 Pass 2 的目标是 60dB 而非 50dB，这将显著影响最终结果。未来可以研究更有导向性的参数排序策略，例如基于参数敏感性分析，优先调整对当前瓶颈性能影响最大的参数。

**超参数的自适应调整：**当前框架中的贪心跳转阈值、微扰步长等超参数是基于经验设定的。这些固定值可能并非在所有优化阶段或所有电路上都是最优的。未来的工作可以探索开发自适应调整策略，让算法能够根据当前的优化状态（例如，离可行域的距离、收敛速度等）动态调整这些超参数，以提升框架的泛化能力和鲁棒性。

**功能性对称的识别与应用：**WL 算法仅能识别纯粹的拓扑对称，无法理解电路的功能模块。例如，电路设计中常见的偏置电路，其内部某些 MOS 管即使拓扑结构不完全相同，也常常被设计为相同尺寸。识别这些“功能性对称”需要引入额外的电路先验知识，这超出了标准 WL 算法的能力范畴。未来可以探索将专家规则或机器学习方法与图算法相结合，以挖掘更深层次的参数绑定关系，进一步降低优化维度。

**评估函数的构建是引导优化的核心，**当前版本虽然有效，但在惩罚机制的设计上仍有较大提升空间。目前，我们对四个硬性约束（增益、相位裕度、增益裕度和电流）采用的是一种等权重的线性惩罚策略：即将各性能指标的归一化违例量直接相加，构成总惩罚值。这种方法的简化之处在于，它隐含了所有约束同等重要的假设，并且惩罚力度与违例程度呈线性关系。然而，在实际模拟电路设计中，不同性能指标的优先级和敏感度差异悬殊。例如，相位裕度（Phase Margin）的轻微不足可能直接导致电路振荡，其后果远比增益

（Gain）指标上同等比例的缺失更为严重。

未来的改进方向可以聚焦于构建一个更精细化的多目标评估体系。首先，可以引入非均匀的加权惩罚，为直流增益这类约束分配更高的权重系数，从而引导优化器优先解决致命问题。其次，可以探索非线性惩罚函数，例如对违例量施加二次方或指数级惩罚，当某个参数严重偏离目标时，其惩罚值会急剧增大，形成更陡峭的“惩罚壁垒”，这有助于算法更快地从无效区域收敛。同样，对于 UGB 和面积这两个优化目标，当前固定的权重也限制了设计的灵活性。未来可将其设计为可配置的超参数，允许用户根据具体应用场景来灵活调整面积与速度之间的权衡。

## 五、参考文献

- [1] K. Settaluri, Z. Liu, R. Khurana, A. Mirhaj, R. Jain, and B. Nikolic, "Automated design of analog circuits using reinforcement learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 9, pp. 2794-2807, 2022.
- [2] R. A. Rutenbar, G. G. E. Gielen, and B. A. A. Antao, "Computer-Aided Design of Analog Integrated Circuits and Systems," *IEEE Press*, 2002.
- [3] B. L. Douglas, "The Weisfeiler-Lehman test," *arXiv preprint arXiv:1312.6391*, 2013.